

## Kapitel 4: Effizienz

Amdahls Gesetz | Code-Optimierungen | Cache & Pipeline

### 1 Was ist Effizienz?

#### Definition 4.1 — Effizienz des Codes

$$\text{Effizienz} = \underbrace{\text{Algorithmische Effizienz}}_{\text{Komplexitätsklasse}} + \underbrace{\text{Effiziente Implementierung}}_{\text{Low-Level-Optimierung}}$$

Ein Algorithmus ist effizient, wenn er **so schnell wie nötig** ist — nicht unbedingt so schnell wie möglich. Die Laufzeit hängt ab von: Eingabegröße, Datenstruktur, Hardware und Implementierung.

#### Achtung

**Richtige Reihenfolge:** Erst korrekt programmieren und debuggen — *dann* optimieren. Nie blind optimieren! Immer zuerst Profiler einsetzen, um den echten Bottleneck zu finden.

#### Vorgehen 4.2 — Profilergestützte Optimierung

1. **Profiler verwenden:** Herausfinden, wo die Zeit tatsächlich verbraucht wird.
2. **Kritische Teile optimieren:** Typischerweise  $< 20\%$  des Codes, die  $> 80\%$  der Laufzeit ausmachen (**80/20-Regel**, Pareto-Prinzip).

Intuitionen über Bottlenecks sind häufig falsch — ohne Profiler nie optimieren.

### 2 Amdahls Gesetz

#### Satz 4.3 — Amdahls Gesetz

Sei  $f_i \in [0, 1]$  der **Anteil** eines Programnteils an der Gesamtlaufzeit und  $f_{sp} > 1$  sein **Beschleunigungsfaktor**. Der **Gesamtspeedup** beträgt:

$$\text{Speedup} = \frac{1}{(1 - f_i) + \frac{f_i}{f_{sp}}}$$

**Maximaler Speedup** (auch mit unendlicher Beschleunigung des Teils):

$$\text{Speedup}_{\max} = \lim_{f_{sp} \rightarrow \infty} \frac{1}{(1 - f_i) + \frac{f_i}{f_{sp}}} = \frac{1}{1 - f_i}$$

**Kernaussage:** Ein *kleiner* Programnteil, der *viel* Zeit einnimmt, bringt beim Optimieren viel. Ein schneller Programnteil, egal wie stark beschleunigt, bringt wenig.

#### Beispiel 4.4 — Amdahl: Welche Optimierung lohnt mehr?

**Szenario:** Zwei Alternativen zur Optimierung:

| Option                         | $f_i$ | $f_{sp}$    | Gesamtspeedup                        |
|--------------------------------|-------|-------------|--------------------------------------|
| Optimierung $f$ : großer Teil  | 80 %  | $2\times$   | $\frac{1}{0,2 + 0,4} \approx 1,67$   |
| Optimierung $g$ : kleiner Teil | 20 %  | $100\times$ | $\frac{1}{0,8 + 0,002} \approx 1,25$ |

**Ergebnis:** Optimierung  $f$  (großer Anteil, moderater Speedup) bringt mehr als Optimierung  $g$  (kleiner Anteil, extremer Speedup).  $\Rightarrow$  **Erst den größten Anteil optimieren!**

**Weiteres Beispiel:**  $f_i = 40\%$ ,  $f_{sp} = 2$ :

$$\text{Speedup} = \frac{1}{0,6 + 0,2} = 1,25$$

Maximaler Speedup:  $\frac{1}{1-0,4} = \frac{1}{0,6} \approx 1,67$  (auch mit unendlicher Beschleunigung).

### 3 Einfache Code-Optimierungstechniken

#### Technik 4.5 — Drei klassische Code-Optimierungen

##### (a) Eliminierung redundanten Codes

Doppelt berechnete Ausdrücke vermeiden — Zwischenergebnis in Variable speichern.

##### (b) Common Subexpression Elimination (CSE)

Gleiche Teilausdrücke, die mehrfach vorkommen, einmal berechnen und wiederverwenden.

##### (c) Loop Invariant Code Motion

Ausdrücke, die sich in der Schleife *nicht* ändern, aus der Schleife herausziehen.

#### Beispiel 4.6 — Alle drei Techniken in Python

```
# (a) Redundanz: Diskriminante nur einmal berechnen
# Schlecht:
x1 = (-p + (p**2 - 4*q)**0.5) / 2
x2 = (-p - (p**2 - 4*q)**0.5) / 2    # p**2-4*q doppelt!

# Gut:
disc = (p**2 - 4*q)**0.5
x1 = (-p + disc) / 2
x2 = (-p - disc) / 2

# (b) CSE: gemeinsamen Teilausdruck auslagern
# Schlecht:
y = a[i]*b[j] + c[i]*b[j]          # b[j] doppelt!
# Gut:
bj = b[j]
y = a[i]*bj + c[i]*bj

# (c) Loop Invariant: len(a) aus Schleife raus
# Schlecht:
for i in range(len(a)):              # len(a) in jeder Iteration!
```

```

    process(a[i])
# Gut:
n = len(a)
for i in range(n):
    process(a[i])

```

## 4 Optimierungen der inneren Schleife

### 4.1 Speicherallokation vermeiden

#### Technik 4.7 — Speicherallokation bündeln

**Problem:** Einzelne Speicherallokationen (`malloc/new`/Python-Objekte) sind teuer, da die Freelist (verkettete Liste freier Blöcke) linear durchsucht werden muss:  $\mathcal{O}(n)$  im schlechtesten Fall.

**Lösung:** Einmal einen großen Block allozieren und intern selbst verwalten (eigene Freelist im Array). Wiederverwendung von Objekten statt ständigem Allozieren/Freigeben.

### 4.2 Funktionsaufrufe vermeiden

#### Technik 4.8 — Funktionsaufruf-Overhead reduzieren

**Problem:** Jeder Funktionsaufruf erfordert: Stack-Frame aufbauen, Parameter übergeben, Rücksprungadresse speichern, Rückgabewert übergeben, Frame abbauen.

**Lösungen:**

- `inline`-Funktionen (C/C++): Compiler fügt Funktionskörper direkt ein.
- Makros (C): Textersetzung, kein Aufruf.
- Iteration statt Rekursion: spart wiederholten Stack-Aufbau.

### 4.3 Arithmetische Tricks

#### Technik 4.9 — Schnelle Arithmetik

| Langsam                          | Schnell                   | Grund                    |
|----------------------------------|---------------------------|--------------------------|
| <code>x / 2</code>               | <code>x &gt;&gt; 1</code> | Bit-Shift statt Division |
| <code>x % 2</code>               | <code>x &amp; 1</code>    | Bitmaske statt Modulo    |
| <code>x**3.0 / pow(x,3.0)</code> | <code>x*x*x</code>        | kein Gleitkomma-Overhead |
| <code>x *= 2</code>              | <code>x += x</code>       | Multiplikation vermeiden |
| <code>x = x + a</code>           | <code>x += a</code>       | kein temporäres Objekt   |

**Achtung:** Bit-Shifts nur für nicht-negative Ganzzahlen korrekt!

## 5 Prozessor-Pipeline ausnutzen

### Definition 4.10 — CPU-Pipeline und Branch Misprediction

Moderne CPUs führen Befehle **pipelined** aus: Fetch → Decode → Execute → Write-Back laufen gleichzeitig für verschiedene Befehle.

**Branch Misprediction:** Bei einer **if/else**-Verzweigung spekuliert die CPU einen Pfad. Ist die Vorhersage falsch, wird die Pipeline *geleert* ( $\approx 10\text{--}20$  Takte verschwendet).

**Gegenmaßnahmen:**

- Verzweigungen in der inneren Schleife *reduzieren* oder *umstrukturieren*.
- **memset** für blockweise Nullsetzung statt Schleife mit Bedingung.
- Bedingte Ausdrücke ohne Sprung (z. B. `x = (cond) ? a : b` in C).
- Typkonvertierungen vermeiden — erzeugen Datenabhängigkeiten zwischen Befehlen.

## 6 Cache-Effizienz

### Definition 4.11 — Cache-Hierarchie und Lokalität

**Zugriffszeiten (typisch):**

| Speicher | Latenz                          | Faktor vs. L1          |
|----------|---------------------------------|------------------------|
| Register | $< 1\text{ ns}$                 | $1\times$              |
| L1-Cache | $\sim 1\text{ ns}$              | $1\times$              |
| L2-Cache | $\sim 5\text{ ns}$              | $5\times$              |
| RAM      | $\sim 50\text{--}100\text{ ns}$ | $50\text{--}100\times$ |
| SSD      | $\sim 100\text{ }\mu\text{s}$   | $100\,000\times$       |

**Spatial Locality:** Auf Daten zugreifen, die im Speicher *nahe beieinander* liegen — sie werden zusammen in eine Cache-Line geladen.

**Temporal Locality:** Kürzlich genutzte Daten *bald wieder* nutzen — solange sie noch im Cache sind.

### Beispiel 4.12 — Cache-Lokalität: Matrix-Traversierung

```
import numpy as np
A = np.zeros((1000, 1000))

# Gut: zeilenweise (row-major, zusammenhaengender Speicher)
for i in range(1000):
    for j in range(1000):
        A[i][j] = i + j    # Cache-freundlich

# Schlecht: spaltenweise (jeder Zugriff springt 1000 Eintraege)
for j in range(1000):
    for i in range(1000):
        A[i][j] = i + j    # Cache-Miss in jeder Iteration
```

Arrays (zusammenhängender Speicher) haben gute Cache-Lokalität. Verkettete Listen (Knoten verteilt im Heap) haben *schlechte* Cache-Lokalität.

### Achtung

**Komplexität  $\neq$  Praxisgeschwindigkeit!** Ein Algorithmus mit besserer asymptotischer Komplexität kann in der Praxis *langsamer* sein, wenn er schlechtere Cache-Lokalität hat. Beispiel: Linked-List-Merge vs. Array-Sort bei kleinen  $n$ .

## 7 Python-spezifische Hinweise

### Technik 4.13 — Vektorisierung mit NumPy

Explizite `for`-Schleifen in Python sind sehr langsam (Interpreter-Overhead pro Iteration). `numpy` ruft intern optimierte C-/Fortran-Implementierungen (BLAS, LAPACK) auf.

### Beispiel 4.14 — Python-Schleife vs. NumPy

```
import numpy as np

n = 10_000_000
a = list(range(n))
b = list(range(n))

# Langsam: explizite Python-Schleife
c = [a[i] + b[i] for i in range(n)]    # ~1 Sekunde

# Schnell: NumPy-Vektorisierung
a_np = np.array(a)
b_np = np.array(b)
c_np = a_np + b_np                    # ~10 ms (100x schneller!)
```

Faustregel: **Immer vektorisieren**, wenn möglich — Schleifen über große Arrays gehören in NumPy.

## 8 Übersicht: Optimierungsebenen

| Ebene         | Technik                          | Potenzial | Aufwand |
|---------------|----------------------------------|-----------|---------|
| Algorithmus   | bessere Komplexitätsklasse       | sehr hoch | hoch    |
| Datenstruktur | Array statt Liste (Cache)        | hoch      | mittel  |
| Code          | CSE, Loop-Invarianten, Redundanz | mittel    | gering  |
| Speicher      | Allokation bündeln, Freelist     | hoch      | mittel  |
| CPU           | Pipeline, Branches reduzieren    | mittel    | gering  |
| Cache         | Spatial/Temporal Locality        | hoch      | mittel  |
| Sprache/Lib   | NumPy, inline, Iteration         | hoch      | gering  |

## Prüfungsscheckliste — Kapitel 4: Effizienz

---

- ☐ Zwei Komponenten von Effizienz nennen: algorithmische Komplexität + Low-Level-Implementierung. → *Def. 4.1*
- ☐ Richtige Optimierungsreihenfolge: erst korrekt, dann Profiler, dann Bottleneck optimieren. → *Vorgh. 4.2*
- ☐ 80/20-Regel (Pareto) erklären: welcher Anteil des Codes verursacht den Großteil der Laufzeit? → *Vorgh. 4.2*
- ☐ Amdahls Gesetz: Formel aufschreiben, Grenzwert  $\frac{1}{1-f_i}$  herleiten. → *Satz 4.3*
- ☐ Amdahl-Beispielrechnung durchführen: Speedup ausrechnen, Optionen vergleichen. → *Bsp. 4.4*
- ☐ Wann lohnt Optimierung? Anteil  $f_i$  entscheidet — nicht der Speedup-Faktor allein. → *Bsp. 4.4*
- ☐ Drei Code-Optimierungen benennen und je ein konkretes Beispiel angeben: Redundanz, CSE, Loop-Invariant. → *Techn. 4.5/4.6*
- ☐ Warum ist Speicherallokation langsam? (Freelist, lineare Suche) — Gegenmaßnahme nennen. → *Techn. 4.7*
- ☐ Warum sind Funktionsaufrufe teuer? Stack-Frame erklären. Gegenmaßnahmen: inline, Iteration. → *Techn. 4.8*
- ☐ Arithmetische Tricks: Bit-Shift statt Division/Modulo, `x*x*x` statt `pow`. → *Techn. 4.9*
- ☐ CPU-Pipeline erklären: was ist Branch Misprediction, wie viele Takte gehen verloren? → *Def. 4.10*
- ☐ Gegenmaßnahmen gegen Branch Misprediction: Verzweigungen reduzieren, `memset`. → *Def. 4.10*
- ☐ Cache-Hierarchie: Latenzunterschiede L1/L2/RAM ungefähr angeben. → *Def. 4.11*
- ☐ Spatial vs. Temporal Locality erklären — je ein Beispiel. → *Def. 4.11*

- ☐ Warum hat zeilenweise Matrix-Traversierung bessere Cache-Lokalität als spaltenweise? → *Bsp. 4.12*

---

  - ☐ Warum können Arrays trotz gleicher Komplexität schneller sein als Listen? (Cache-Lokalität) → *Achtung, §6*

---

  - ☐ Python: Warum ist NumPy drastisch schneller als explizite Schleifen? → *Bsp. 4.14*  
Beispiel angeben.
- 

**Literatur:** Wiki-Skript (Prof. Kothe)